JavaScript Error Handling Guide

Comprehensive Q&A covering errors, try/catch semantics, propagation, and async boundaries.

Core Conceptual Questions (Part 1)

1. Difference between Error, TypeError, ReferenceError, SyntaxError, RangeError, URIError, and AggregateError? Give a practical scenario for each.

> - `Error`: Base object for custom or generic errors (e.g., throwing a generic app error).
> - `TypeError`: A value is not of the expected type (e.g., calling `undefined()` or `null.map()`).
> - `ReferenceError`: Referencing an undeclared/uninitialized variable (e.g., accessing a `let` variable before initialization).
> - `SyntaxError`: Invalid JavaScript syntax (e.g., parsing invalid JSON with `JSON.parse("{ bad }")`).
> - `RangeError`: A numeric variable or parameter is outside its valid range (e.g., `new Array(-1)` or excessive recursion).
> - `URIError`: Improper use of global URI handling functions (e.g., `decodeURIComponent("%")`).
> - `AggregateError`: Represents multiple errors wrapped in a single object (e.g., when `Promise.any()` rejects, it returns an AggregateError of all underlying promise failures).

2. How do throw, try, catch, and finally work together? What happens when return or throw is executed inside finally?

> `try` encloses code that might fail; `catch` executes if an error is `throw`n; `finally` executes unconditionally after the `try/catch` blocks conclude.
>
> **Crucial Behavior:** If a `return` or `throw` statement is executed inside a `finally` block, it **overrides** any pending `return` or `throw` from the `try` or `catch` blocks. It acts as the final exit mechanism for the function.

3. How would you create a custom error class extending Error? Explain instanceof, name, message, stack, and cause.

> ```
> class DatabaseError extends Error {
>   constructor(message, options) {
>     super(message, options);
>     this.name = "DatabaseError";
>   }
> }
> throw new DatabaseError("Connection failed", { cause: originalError });
> ```
>
> - `instanceof`: Will correctly return `true` for `err instanceof DatabaseError` and `err instanceof Error` via prototype chaining.
> - `name`: Overridden to identify the error type.
> - `message`: The human-readable string passed to `super()`.
> - `stack`: Automatically generated trace of the execution context.
> - `cause`: Introduced in ES2022, allows wrapping the original error that triggered this failure, preserving context.

Core Conceptual Questions (Part 2)

4. How does error propagation work across nested function calls? When should you catch an error, rethrow it, or allow it to propagate?

> Errors "bubble up" the call stack synchronously until they hit the first matching `catch` block. If uncaught, they crash the process.
>
> - **Catch:** When the current layer can completely handle and recover from the failure (e.g., retrying, showing a UI warning).
> - **Rethrow:** When you need to log it or wrap it in a higher-level custom error (using `cause`) to add contextual abstraction, but cannot fully resolve it.
> - **Propagate:** When the current function shouldn't know how to handle the error, leaving it to a global handler or higher-level orchestrator.

5. Why does synchronous try/catch not catch an unawaited promise rejection? How do Promise handlers differ?

> Synchronous `try/catch` finishes executing before any asynchronous microtasks run. If a Promise rejects without an `await`, the synchronous `try` block has already concluded, resulting in an unhandled rejection.
>
> - `async/await`: Allows `try/catch` to pause execution and catch rejections synchronously.
> - `.catch()`: Attaches a handler explicitly to a Promise chain.
> - `Promise.all()`: Fail-fast; rejects immediately if *any* promise rejects.
> - `Promise.any()`: Succeed-fast; ignores individual rejections unless *all* promises reject (resulting in an `AggregateError`).

6. What are React error boundaries, what errors can they catch, and how do they differ from try/catch?

> React Error Boundaries are class components implementing `componentDidCatch` or `static getDerivedStateFromError`. They act like a declarative `catch` block for the component tree below them, displaying a fallback UI instead of crashing the tree.
>
> **Difference:** They *only* catch errors during rendering, lifecycle methods, and constructors of child components. They **do not** catch errors in event handlers, asynchronous code (e.g., `setTimeout`), server-side rendering, or errors thrown inside the boundary itself (this requires traditional `try/catch`).

7. What is the difference between fail-fast and graceful degradation? How would you decide which to use?

> **Fail-fast:** The application immediately stops execution and throws an error upon an unexpected condition. Used for initialization errors (e.g., missing database URI) or critical data invariants to prevent data corruption.
> **Graceful degradation:** The app handles the failure and provides a fallback (e.g., serving cached data if an API is down). Used for non-critical features to ensure the core user experience survives partial system outages.

Core Concepts & Output Analysis

8. How would you design centralized error handling for a Node.js/Express API?

> 1. **Custom Error Class:** Create an `AppError` class containing an HTTP `statusCode` and an `isOperational` boolean flag.
> 2. **Middleware:** Use an Express global error-handling middleware `(err, req, res, next)` at the end of the pipeline.
> 3. **Classification:** Inside the middleware, check `isOperational`. If true (e.g., 404, 400), send the message safely to the client. If false (programming bugs, 500), send a generic "Internal Server Error" to prevent leaking stack traces or sensitive credentials.
> 4. **Logging:** Send detailed traces of all 500-level errors to a centralized logger (like Winston or Pino).

9. finally and return precedence

```
function test() {
  try { throw new Error("A"); }
  catch (error) { return "B"; }
  finally { return "C"; }
}
console.log(test());
```

> **Output:** `"C"`
> **Why:** The `catch` block catches the error and prepares to return `"B"`. However, the `finally` block executes before the return is materialized. Because `finally` has its own `return "C"` statement, it overwrites the pending return value from the catch block.

10. Error propagation and execution order

```
function first() {
  console.log("A");
  throw new TypeError("B");
}
function second() {
  try { first(); console.log("C"); }
  catch (error) {
    console.log(error.name);
    throw new Error("D", { cause: error });
  }
  finally { console.log("E"); }
}
try { second(); }
catch (error) {
  console.log(error.message);
  console.log(error.cause.message);
}
console.log("F");
```

> **Output:** `A`, `TypeError`, `E`, `D`, `B`, `F`
> **Why:** `first()` logs A and throws. The try in `second()` catches it (skipping C), logs `error.name` (TypeError), and throws a new Error D. Before propagating Error D out of `second()`, the `finally` block runs and logs E. Then the outer catch intercepts D, logs `error.message` (D) and its wrapped `cause.message` (B). Finally, synchronous execution continues and logs F.

Output-Based Questions (Continued)

11. Promise rejection and synchronous try/catch

```
async function test() { throw new Error("Failed"); }
try {
  test();
  console.log("A");
} catch (error) { console.log("B"); }
console.log("C");
```

> **Output:** `A`, `C` (followed by an UnhandledPromiseRejection warning).
> **Why:** An `async` function always returns a Promise. Because we did not `await test()`, it synchronously returns a rejected Promise and execution continues normally. The synchronous `try/catch` does not detect promise rejections, skipping the catch block entirely.

12. await, rejection handling, and finally

```
async function task() {
  try { await Promise.reject(new TypeError("Failure")); console.log("A"); }
  catch (error) { console.log(error.name); return "B"; }
  finally { console.log("C"); }
}
task().then(console.log);
console.log("D");
```

> **Output:** `D`, `TypeError`, `C`, `B`
> **Why:** `task()` suspends on the `await`, queueing a microtask. Synchronous execution continues, logging `D`. Then the microtask runs: the await rejects, jumping to catch (logging `TypeError` and preparing to return B). `finally` runs, logging `C`. The promise resolves with B, triggering `.then(console.log)` which logs `B`.

13. Promise.any() and AggregateError

```
Promise.any([
  Promise.reject(new TypeError("A")),
  Promise.reject(new RangeError("B"))
]).catch(error => {
  console.log(error instanceof AggregateError);
  console.log(error.errors.map(e => e.name).join(","));
});
```

> **Output:** `true`, `TypeError,RangeError`
> **Why:** `Promise.any()` only rejects if *all* provided promises reject. When that happens, it throws an `AggregateError`. The `.errors` property contains an array of the underlying errors in the order they were provided in the iterable.

Output-Based Questions (Continued)

14. Custom error inheritance and classification

```
class AppError extends Error {
  constructor(message) {
    super(message);
    this.name = "AppError";
  }
}
class NotFoundError extends AppError {}

try { throw new NotFoundError("Missing"); }
catch (error) {
  console.log(error instanceof NotFoundError);
  console.log(error instanceof AppError);
  console.log(error instanceof Error);
  console.log(error.name);
}
```

> **Output:** `true`, `true`, `true`, `AppError`
> **Why:** `NotFoundError` does not have its own constructor, so it inherits `AppError`'s constructor. The `instanceof` checks are all true due to prototype chaining. Because the parent constructor hardcodes `this.name = "AppError"`, that name is applied to the `NotFoundError` instance as well.

15. Can finally suppress an error?

```
function execute() {
  try { throw new Error("Original"); }
  finally {
    console.log("Cleanup");
    return "Success";
  }
}
try { console.log(execute()); }
catch (error) { console.log(error.message); }
```

> **Output:** `Cleanup`, `Success`
> **Why:** Yes, a `finally` block can suppress an error. When `execute()` throws an error in the try block, the `finally` block intercepts it. By explicitly executing a `return` statement inside `finally`, the pending error throw is discarded, and the function successfully returns `"Success"`. The outer catch block is never triggered.